



Date: 04.05.2026

Time: 10.30 a.m – 11.30 a.m

INSTRUCTIONS TO CANDIDATES

- This paper contains **one (1) question** and **three (3) pages**.
- Paper duration is **one hour**.
- Create a folder named **“Practical Test – OOP”** in either the **D:** or **E:** drive. Inside that folder, **create another folder** using **your registration number** as the name. **Set your project path to the folder named with your registration number.**
- Name your project as **“OOP, _(Underscore) and your registration number, e.g. OOP_123456789”** and for location browse for the location where your folder is saved.
- Always keep saving your code so there will be no issue if there is an electricity break down.
- While coding, you **must copy and paste** your **code into a Word document**.
- Save the Word document inside the **123456789** folder and **rename it with your Registration Number (123456789.docx)**.
- At the end of the practical test, **export the Word document as a PDF** and **save it in the same 123456789 folder**.
- At the end of the test, please check whether the **123456789 folder contains the following:**
 - OOP_123456789 (project folder)
 - Word document (123456789.docx)
 - PDF file (123456789.pdf)

Scenario: Smart Retail Store Billing System

In a retail store system, every **Product** has a **final productID** and a **final storeName** that cannot be changed once the product is registered. Each product also stores its **productName** and **price**.

There are two types of products: **FoodItem** and **ElectronicItem**, which inherit from the Product class. The system allows price calculation in different ways. Each type of product displays its details differently.

All products that can be billed must follow a common interface. The base class should be designed using abstraction. The system must also ensure proper encapsulation using access modifiers.

1. Identify and create the parent class for the system.
2. Declare the following attributes in the parent class:
 - productID (final)
 - storeName (final)
 - productName
 - price
3. Write a constructor to initialize all variables.
4. Make the parent class an **abstract class** and:
 - Declare an abstract method displayDetails()
5. Create two subclasses:
 - FoodItem
 - ElectronicItem
6. Override the displayDetails() method in both subclasses.
7. Implement **method overloading** in the parent class:
 - calculatePrice() → returns price
 - calculatePrice(double discount) → returns price – discount
8. Create an interface Billable with the following method:
 - printBill()
 - Implement this interface in one subclass (FoodItem).

9. Apply encapsulation:
- Make variables private
 - Implement getter and setter methods for price

10. Create a Test (main) class and:

- Create one FoodItem object:
 - productID: "P101"
 - storeName: "Cargills"
 - productName: "Bread"
 - price: 150
- Create one ElectronicItem object:
 - productID: "P202"
 - storeName: "Singer"
 - productName: "Electric Kettle"
 - price: 5000

11. Call the following methods:

- calculatePrice()
- calculatePrice(50)
- displayDetails()
- printBill() (for FoodItem object)

Expected Output :

```
---- Food Item ----
Price: 150.0
Price after discount: 100.0
Food Item Details:
Product Name: Bread
Bill printed successfully.

---- Electronic Item ----
Price: 5000.0
Price after discount: 4950.0
Electronic Item Details:
Product Name: Electric Kettle
```

***** END OF THE PAPER *****